

KAPITEL 6: PHP

LERNZIELE

- Erklären können, wie Webseiten serverseitig erzeugt werden können und wie externe Generatoren (z.B. PHP) dabei eingesetzt werden.
- Die PHP-Grundkonzepte kennen und zur Realisierung kleiner Webanwendungen einsetzen können.
- Formulare entwickeln können, die Daten serverseitig mit PHP verarbeiten und Antwortseiten generieren.
- Das MVC2-Muster erklären können und auf eigene kleine Webanwendungen übertragen können.

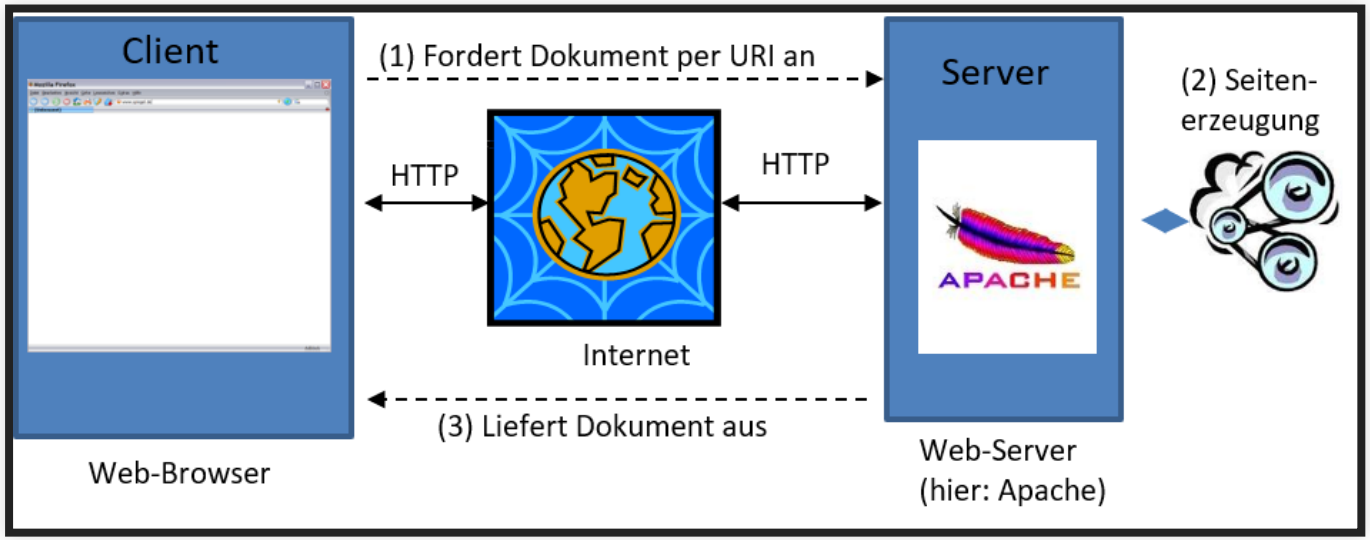
6.1 SERVERSEITIGE HTML-GENERIERUNG



Statische Webseiten in HTML werden - einmal erstellt - immer mit dem selben Zustand an den Browser gesendet.

Webseiten wie ein Shop oder ein Blog ändern sich je nach Nutzereingaben oder verfügbarer Blogeinträge und Kommentare. Entsprechend müssen Sie regelmäßig neu erzeugt werden. Zum Beispiel mit einer HTTP-Anfrage. In diesem Fall wird von dynamischen Webseiten gesprochen.

Dynamische Webseiten präsentieren Inhalte aus Dateien, Datenbanken oder anderen Quellen. Diese HTML-Seiten werden in der Regel auf dem Webserver erzeugt, bevor sie zum Browser geschickt werden.

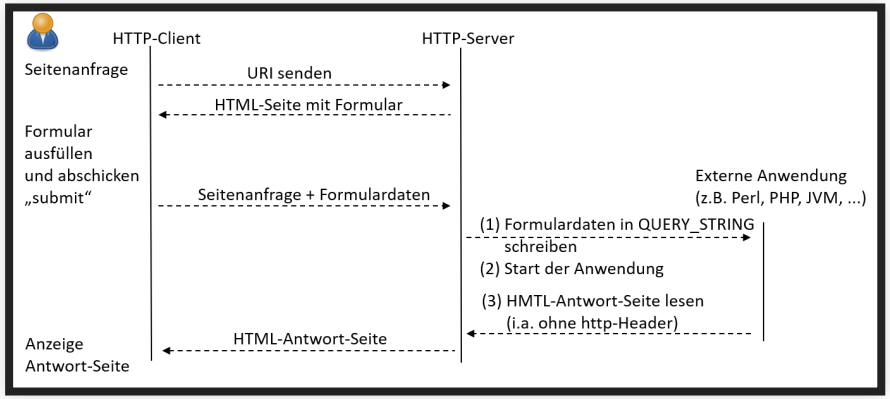


CGI: Standard-Interface

Zur Flexibilisierung der Seitenerzeugung wurde das **Common Gateway Interface (CGI)** definiert, welches die Schnittstelle zwischen dem Webserver (HTTP-Server) und einem externen Programm (=HTML-Generator) standardisiert.

- Das externe Programm wird vom Webserver beauftragt, eine HTML-Seite zu erzeugen.
- Die Festlegungen sind einfach:
    1. Die erzeugte Seite wird über die Standardausgabe als Text zurückgegeben, d.h. der Webserver liest den Output des Generators.
    2. Beim Start wird für den Generator eine Umgebungsvariable "QUERY_STRING" belegt. Diese enthält die Eingabewerte für die Generierung (z.B. Formulardaten).

Ablauf der HTML-Generierung



- Nach dem "Submit" wird die Anfrage ("action"-Attribut in form-Tag) zusammen mit den Formular-Inputs an den Server gesendet.
- Der Server erkennt anhand der Extension der angefragten Seite (z.B. `.php`), welche externe Anwendung zu starten ist.
- Die externe Anwendung schreibt den Text der neuen HTML-Seite auf ihre Standardausgabe, die vom Webserver gelesen wird.
- Der Webserver sendet die Seite um Header ergänzt als HTTP-Nachricht an den Browser.

Multithreaded Server

Das Starten eines eigenen Prozesses zur Seitengenerierung ist bei sehr vielen Aufrufen zu langsam (hoher Betriebssystem-Overhead für Prozess-Start).

Entsprechend wird in solchen Fällen statt eines Prozesses ein Thread innerhalb des Webservers gestartet.

Threads haben keinen eigenen Prozess-Kontext und können somit wesentlich schneller erzeugt und wieder aufgelöst werden.

Technologien für serverseitige HTML-Generierung (Liste nicht vollständig!)

Technologie	Charakteristik	Bewertung
PHP	PHP-Skript erzeugt HTML-Seite	Wartbarkeit nicht gut, keine Trennung von View und Logik/Model; verbesserte Wartungsfeundlichkeit durch Template-Mechanismus (z.B. Smarty)
Java Server Pages (JSP)	Trennung von View und Logik/Daten	wartbar, jedoch unflexibel
Java Server Faces (JSF)	eigene Tags definierbar; AJAX-Controls	Flexibler, gut modularisiert
ASP.NET (Microsoft)	Ähnlich wie JSF, VB bzw. C#	Wie JSF, sehr gute Integration in Windows-Umgebung

Technologie	Charakteristik	Bewertung
NodeJS, ExpressJS und Liquid	JavaScript als Backend-Programmiersprache, HTTP/REST-Framework für die Anfragen und eine Template-Sprache zur Trennung von Logik und View	Erlaubt auf Server-Seite mit der selben Sprache zu arbeiten wie im Browser

Beispiel mit PHP

Die Problematik vieler Ansätze wird durch das folgende PHP-Beispiel verdeutlicht. Es generiert eine Tabelle mit Quadratzahlen ("1 | 1" bis "9 | 81").

```
<table>
<?php
    for ($i = 1; $i < 10; ++$i) {
        echo '<tr><td>'.$i.'</td><td>'.$i*$i.'</td></tr>';
    }
?>
</table>
```

- Hier beginnt die Tabelle.
- Der PHP-Interpreter erzeugt über "echo" den HTML-Code für die tr's und td's.
- HTML-Code und PHP-Logik-Code sind vermischt, insgesamt ergibt sich eine schlechte Wartbarkeit und Lesbarkeit.
- Moderne Ansätze trennen View(=HTML)-Code und Logik-Code (hier: PHP), um die Wartbarkeit zu verbessern.

Folgende Animation zeigt, wie die Generierung einer Webseite mit PHP funktioniert



ÜBUNG: SERVERSEITIGEN GENERIERUNG

Welche der folgenden Aussagen über serverseitige Generierung (wie mit PHP) sind korrekt?

- [ ] Eine Webseite kann ohne HTTP-Request generiert und zum Client / Browser gesendet werden
- [ ] Serverseitige Skriptsprachen wie PHP können auf Ressourcen zugreifen, die für Client-seitige Skripte nicht direkt verfügbar sind
- [ ] Bei einem HTTP-Request wird dynamisch eine HTML-Seite generiert
- [ ] Anwendungs-Code (z.B. PHP) wird auf dem Client / Browser ausgeführt

ÜBUNG: ABLAUF EINER HTTP-ANFRAGE MIT PHP

Erklären Sie kurz, wie PHP mit HTTP-Requests interagiert und welche Rolle es im typischen Ablauf einer Webanfrage spielt.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt



6.2 PHP SPRACHKONZEPTE

PHP steht für Personal Homepage Tools oder auch PHP: Hypertext Preprocessor (lt. W3C) und ist eine objektorientierte Skripsprache zur serverseitigen Generierung von Webseiten.

PHP ist sehr verbreitet, die Gründe dafür liegen in der leichten Erlernbarkeit sowie in der Vielzahl an Funktionsbibliotheken (z.B. für DB-Anbindung, Dokument- bzw. Grafikgenerierung usf.).



PHP Version

- Seit Dezember 2020 gibt es PHP in der Version 8.
- Erst ab Version 5 wurde PHP objektorientiert, d.h. es gibt in PHP
  - analog zu Java Konstruktoren und analog zu C++ Destruktoren,
  - Sichtbarkeiten (public, protected, private),
  - Klasseneigenschaften (Schlüsselwort "static"),
  - Abstrakte Klassen,
  - Interfaces und Reflection (Abfrage von Objekteigenschaften zur Laufzeit).
- Die aktuell verwendete Version erhält man durch Ausführung des Skripts

```php
<?php phpinfo(); ?>
```



Wie lernt man PHP?



- Es gibt eine Vielzahl guter Bücher zu PHP, man kann PHP aber auch gut "online" lernen, z.B. mit dem Tutorial des W3C.
- Das PHP-Handbuch gibt es in deutscher Sprache.
- Viele konkrete Fragen lassen sich schnell durch eine geeignete Internet-Suche (englisch!) klären, z.B. findet man durch "search string in string php" schnell die Funktion *strpos* auf http://php.net/manual/de/function.strpos.php.
- Wie immer ist beim Erlernen einer Programmiersprache das Ausprobieren und Üben am Rechner unerlässlich, daher wird empfohlen, ein fertiges Paket wie XAMPP zu installieren.
- Nach weniger als 5 Minuten hat man einen laufenden Apache-Webserver (zzgl. PHP-Konfiguration) für die Ausführung von PHP-Skripten.
- Als Editor wird wie bisher auch auf Visual Studio Code zurückgegriffen.

HELLO WORLD

Das übliche "Hello World"-Programm sieht in PHP folgendermaßen aus:

```
<html>
<head>
    <title>Hello World mit PHP</title>
</head>
<body>

<?php
    echo 'Hello World!';
    phpinfo();
?>

</body>
</html>
```

Experiment: Ausführung nach dem Download:

- Für die Ausführung ist ein Webserver wie XAMPP notwendig, dieser bietet einen Ordner in dem alle Ressourcen platziert werden müssen
- Skript in diesem Order, dem Webserver Document Root (z.B. C:/xampp/htdocs) speichern (z.B. `example.php`)
- Webserver starten
- Browser mit http://localhost/example.php öffnen

Erläuterungen zum Beispiel

- PHP-Skripte erzeugen HTML-Seiten, müssen also auch die HTML-Grundstruktur definieren.
- Ein PHP-Skript besteht aus "statischen" HTML-Anteilen sowie PHP-Fragmenten.

```php
<?php
    echo "Hello World!";
    phpinfo();
?>
```

- Ein PHP-Fragment bringt dynamische Anteile in die zu erzeugende HTML-Seite und wird mit `<?php` begonnen und mit `?>` beendet.
- Mit `echo` wird die Ausgabe für den Webserver erzeugt.
- Der Aufruf von `phpinfo()` bringt Informationen über die aktuelle PHP-Version in die Seite.

ÜBUNG: PHP EINBETTEN

Wie wird PHP-Code typischerweise in HTML eingebettet?

- ☐ `<!-- PHP: ... -->`
- ☐ `<?php ... ?>`
- ☐ `<php> ... </php>`
- ☐ `<script type="php"> ... </script>`

GENERIERUNG DER AUSGABE MIT ECHO

```php
<?php
    $zeitstempel = time();
    $datum = date('d.m.Y', $zeitstempel);
    $uhrzeit = date('H:i', $zeitstempel);
    echo '<h1>Beispiel für die Funktion echo</h1>';
    echo '<p>Heute ist der ' . $datum .
         ', die Uhrzeit ist ' . $uhrzeit . ' Uhr.</p>';
?>
```

- Mit `echo` kann das Anfügen einer Zeichenkette an das Generierungsergebnis des PHP-Skripts realisiert werden.
- Beide Arten von String-Begrenzern (`"..."` und `'...'`) können verwendet werden.
- Variablen beginnen mit einem `$`-Zeichen (s. nächste Seite).
- Mit dem `.`-Operator können Strings konkateniert werden.

Detaillierte Erläuterungen:

```
$zeitstempel = time();
$datum = date("d.m.Y", $zeitstempel);
$uhrzeit = date("H:i", $zeitstempel);
echo "<h1>Beispiel für die Funktion echo</h1>";
echo "<p>Heute ist der " . $datum .
        ", die Uhrzeit ist " . $uhrzeit . " Uhr.</p>";
```

- Die Funktion `time()` liefert den aktuellen Zeitstempel (Millisekunden seit 1.1.1970), der Wert wird in der Variable "$zeitstempel" gespeichert.
- Variablen müssen in PHP nicht deklariert werden und werden implizit typisiert (vgl. JavaScript).
- Die Funktion `date()` wird verwendet, um das Datum sowie die Uhrzeit aus dem Zeitstempel im String-Format (s. http://php.net/manual/de/function.date.php) zu erhalten.
- Die beiden `echo`-Aufrufe geben die Überschrift sowie Datum und Uhrzeit aus. Durch den `.`-Operator werden konstante und variable Teile des 2. Strings miteinander verbunden.

ÜBUNG: GENERIERUNG

Betrachten Sie den folgenden Skript und bewerten Sie die folgenden Aussagen.

```
<!DOCTYPE html>
<html lang="de">
<head>
    <meta charset="UTF-8">
    <title>Meine erste <?php echo "PHP"; ?>-Seite</title>
</head>
<body>
    <?php
    $name = "Eva";
    $alter = 25;
    $lieblingsfarbe = "Blau";
    ?>
    <p>Name / Alter: <?php echo $name; ?> / <?php echo $alter; ?></p>
    <?php echo "<p>Lieblingsfarbe: $lieblingsfarbe.</p>"; ?>
</body>
</html>
```

Welche Aussagen sind korrekt?

- [ ] Im Browser wird der Text "Name / Alter: Eva / 25" angezeigt.
- [ ] Im Browser wird der Text "Name / Alter: / " angezeigt.
- [ ] Der Text "Lieblingsfarbe: Blau" wird angezeigt.
- [ ] Die Ausgabe wird einen Absatz (p-Elemente) enthalten.
- [ ] Der Seitentitel im Browser-Tab wird "Meine erste PHP-Seite" lauten.

FEHLER UND DEBUGGING

Während JavaScript im Browser ausgeführt wird und damit direkt im Browser mittels Debugger analysiert werden kann, findet die Ausführung von PHP in einem eigenen Prozess über den Webserver statt.

Je nach Konfiguration der PHP-Ausführungsumgebung werden Fehler in Ihren PHP-Skripten nicht immer sichtbar. Damit Fehler angezeigt werden, muss die Fehleranzeige *aktiviert* werden. Eine Möglichkeit ist es, dies über den PHP-Skript selbst zu steuern:

```
error_reporting (E_ALL & ~E_NOTICE & ~E_DEPRECATED);
ini_set ('display_errors', 1);
```

Zeigt alle Fehler, außer Hinweise und Deprecated-Meldungen und sollte möglichst am Anfang platziert werden.

Syntaktische Fehler werden auf diesen Weg nicht sichtbar, da der Interpreter keine Chance hat, diese Anweisungen zur Aktivierung von Fehlerausgaben auszuführen. Alternativ kann in einem Apache + PHP Setup (wie bei XAMPP oder MAMP) im Ordner eine `.htaccess`-Datei platziert werden, die folgenden Inhalt erhält:

```
php_flag display_startup_errors on
php_flag display_errors on
php_flag html_errors on
```

Damit können auch Fehler sichtbar werden, die auf syntaktische Fehker zurückzuführen sind.



Für die Nutzung eines Debuggers ist es notwendig, die PHP-Ausführungsumgebung zu steuern, während die Anfrage läuft. Hierfür gibt es verschiedene Ansätze, die in der Entwicklungsumgebung konfiguriert werden müssen. Eine Möglichkeit ist hierbei der Weg über XDebug.

Mehr Informationen zum Thema Fehler und Debugging

VARIABLEN UND DATENTYPEN

PHP ist wie JavaScript implizit typisiert, d.h. eine Variable erhält ihren Datentyp durch die Belegung mit einem Wert.

Es gibt in PHP folgende Datentypen:

- einfach: `boolean`, `integer`, `double`, `string`
- strukturiert: `array`, `object`
- speziell: `NULL` (Wert NULL), z.B. für Objektreferenzen

Variablen am Beispiel

```php
<?php
    $b = true;
    echo 'Typ von b: '.gettype($b).'<br>';
    $c = 'a';
    echo 'Typ von c: '.gettype($c).'<br>';
    $i = 123;
    echo 'Typ von i: '.gettype($i).'<br>';
    $d = 1.234;
    echo 'Typ von d: '.gettype($d).'<br>';
    $s = 'Hallo Welt!';
    echo 'Typ von s: '.gettype($s).'<br>';
    $nullWert = NULL;
    echo 'Typ von nullWert: '.gettype($nullWert).'<br>';
    echo 'Typ von unbelegt: '.gettype($unbelegt).'<br>';
?>
```



Beachte:

- Variablen in PHP sind Function- oder Global-Scoped, wobei für den globalen Zugriff das Schlüsselwort `global` in der Funktion verwendet werden muss
- Außerhalb von Funktionen definierte Variablen gelten nicht automatisch weiterhin
- Um sie zu nutzen, muss die Variable vor dem Zugriff mit dem Schüsselwort `global` deklariert werden, z.B. `global $meineVariable;`.
- Ohne `global` wird implizit eine Block-lokale Variable mit dem Wert `NULL` angelegt.

ÜBUNG: DATENTYPEN

Welche der folgenden sind primitive/grundlegende Datentypen in PHP?

- [ ] object
- [ ] number
- [ ] array
- [ ] int
- [ ] string
- [ ] boolean
- [ ] long

ÜBUNG: PHP VARIABLEN

Welche der folgenden Variablen-Definitionen sind in PHP gültig?

- [ ] `$2nummer = 42;`
- [ ] `$diesIstEinSehrLangerVariablenName = true;`
- [ ] `$preis1 = 19.99;`
- [ ] `$_alter = 25;`
- [ ] `$meine-variable = "Test";`
- [ ] `$user_name = "Max";`
- [ ] `$währung = "EUR";`

OPERATOREN

PHP bietet eine vergleichbare Sammlung von Operatoren für verschiedene Operationen wie JavaScript.

- Arithmetische Operatoren: `+`, `-`, `*`, `/`, `%`, `++`, `--`
- Zuweisungsoperatoren: `=`, `+=`, `-=`, `*=`, `/=`, `.=`
- Vergleichsoperatoren: `===`, `==`, `<`, `<=`, `>=`, `>`, `!=`, `!==`, `<=>`
- Bitweise Operatoren: `&`, `|`, `^`, `<<`, `>>`, `~`
- String-Operator: `.` (Punkt-Operator für String-Verkettung)

Wie in JavaScript gibt es in PHP die automatische Typkonvertierung (Type Juggling).
PHP versucht automatisch, Variablen in den passenden Kontext zu konvertieren.
Dabei verhält es sich jedoch an einigen Stellen abweichend:

String + Number

```
"5" + 3       // 8 (Konvertiert zu Integer)
"5" . 3       // "53" (String Konkatenation)
"5" * "3"     // 15 (Konvertiert zu Integer)
```

Vergleiche

```
5 == "5"      // true
5 === "5"     // false
[] == false   // true
```

Für sichere Programmierung empfiehlt sich:

- Strikte Vergleichsoperatoren (`===`, `!==`) zu verwenden
- Explizite Typkonvertierung zu nutzen, wenn nötig
- Auf korrekte Datentypen zu achten
- Die PHP-spezifische String-Verkettung mit `.` statt `+` zu verwenden

Ein weiterer spezieller PHP-Operator ist der Null Coalescing Operator `??`:

```php
 // Verwendet 'anonymous' wenn $_GET['name'] null ist
$name = $_GET['name'] ?? 'anonymous';
```

Diese Operatoren bilden die Grundlage für Berechnungen, Vergleiche und Datentypmanipulation in PHP.

ÜBUNG: VERGLEICHSOPERATOREN IN PHP

Welche der folgenden Operatoren führen in PHP eine typstrenge Vergleichsprüfung durch?

- ☐ !==
- ☐ ===
- ☐ ==
- ☐ !=

ÜBUNG: ADDITION IN PHP

Welchen Wert und Datentyp liefert der Ausdruck `"5" + 3`*?*

- [ ] 53 (als integer)
- [ ] 8
- [ ] Error
- [ ] "53" (als string)

KONTROLLFLUSS-STEUERUNG

Blöcke werden in PHP analog zu Java bzw. JavaScript mit geschweiften Klammern definiert.

```
if (<Bedingung>) {
    <Anweisung>;
} else {
    <Anweisung>;
}
```

Kontrollfluss-Steuerung mit `if` und `switch` ist vergleichbar mit Java und JavaScript möglich. Einzeilige Bedinungen können auf die geschweiften Klammern verzichten, ist jedoch keine gute Praxis.



Neben der if-Anweisung gibt es analog zu Java eine `switch`-Anweisung zur Kontrollflussverzweigung abhängig von einem `integer`- oder einem `string`-Wert.

```
$note = 2;
switch ($note) {
    case 1: echo "Sehr gut!"; break;
    case 2: echo "Gut!"; break;
    case 3: echo "Befriedigend"; break;
    case 4: echo "Ausreichend"; break;
    case 5: echo "Mangelhaft"; break;
    case 6: echo "Ungenügend"; break;
    default: echo "Ungültige Note";
}
```

Ein weiteres praktisches Beispiel ist `match` (moderne Alternative seit PHP 8.0):

```php
$note = 2;
$bewertung = match($note) {
    1 => "Sehr gut!",
    2 => "Gut!",
    3 => "Befriedigend",
    4 => "Ausreichend",
    5 => "Mangelhaft",
    6 => "Ungenügend",
    default => "Ungültige Note"
};
echo $bewertung;
```

Die aus Java bekannten Schleifenkonstrukte `while`, `do-while` sowie `for` sind auch in PHP mit analoger Syntax möglich.

```php
$i = 1;
while ($i <= 5) {
    echo "$i ";   // 1 2 3 4 5
    $i++;
}
```

```php
for ($i = 1; $i <= 5; $i++) {
    echo "$i ";   // 1 2 3 4 5
}
```

```php
$i = 1;
do {
    echo "$i ";   // 1 2 3 4 5
    $i++;
} while ($i <= 5);
```

ÜBUNG: KONTROLLSTRUKTUREN IN PHP

Welche der folgenden Kontrollstrukturen gibt es in PHP?

- [ ] match
- [ ] switch
- [ ] for...of (vgl. JavaScript)
- [ ] while
- [ ] if...else
- [ ] do...while
- [ ] for...in (vgl. JavaScript)

ARRAYS

```php
<?php
    $integerArray = array(1, 2, 3);
    $gemischtesArray = array(
        1           => "Ein String",
        "abc"       => 4711,
        "fertig"    => true
    );
    var_dump($integerArray);
?>
```

- Arrays werden mit dem Schlüsselwort array erzeugt.
- Im Gegensatz zu Java sind Arrays in PHP sog. assoziative Arrays, d.h. sie können beliebige Index-Werte haben. Dem Index `abc` wird der Wert 4711 zugewiesen.
- Mit kann `var_dump` eine beliebige Variable ausgegeben werden.
- Der Pfeil `=>` kann verwendet werden, um den Index-Wert und den dazugehörigen Wert zu bezeichnen. Wird er nicht genutzt wie im 1. Beispiel, so werden die Index-Werte von 0 ab nummeriert.

```php
<?php
    $leer = array();
    $leer[] = 18;
    $leer[0] = $leer[0] + 20;
    var_dump($leer);
    echo "<br><br>Gemischtes Array:<br>";
    foreach ($gemischtesArray as $index => $wert) {
        echo $index." =&gt; ".$wert."<br>";
    }
?>
```

- Ohne Initialisierung wird ein leeres Array erzeugt.
- Der `[]`-Zugriff hängt einen Wert an das Ende des Arrays an, wobei hier der nächste freie Integer-Index gewählt wird.
- Der Zugriff über einen Index funktioniert wie in anderen Sprachen auch.
- Mit `foreach` können Arrays bequem iteriert werden, wobei `$index =>` optional ist. Es wird dann nur über die Werte iteriert.

```php
<?php
$matrix = array(array(1, 2), array(3, 4));
echo "<br>Matrix:<br><table>";
for ($zeile = 0; $zeile < count($matrix); ++$zeile) {
    echo "<tr>";
    for ($spalte = 0; $spalte < count($matrix[$zeile]); ++$spalte)
        echo "<td>".$matrix[$zeile][$spalte]."</td>";
    echo "</tr>";
}
echo "</table>";
?>
```

- Arrays können beliebige Werte, also auch wieder Arrays, z.B. zur Abbildung einer 2x2-Matrix, enthalten.
- Die äußere for-Schleife iteriert über ein "Array of Array", wobei "count" die Anzahl der Elemente im Array liefert.
- Die innere for-Schleife iteriert über ein "Array of integer".
- Der Zugriff bei mehreren Dimensionen funktioniert über mehrfach-Anwendung des "[]"-Operators.

Weitere Möglichkeiten zur Array-Erzeugung

```php
<?php
$kurzArray = [1, 2, 3];
$kurzAssoziativ = ['name' => 'Max', 'alter' => 25];

$original = [1, 2, 3];
$kopie = [...$original];
$erweitert = [0, ...$original, 4, 5];

$zahlen = range(1, 10);          // [1, 2, 3, ..., 10]
$buchstaben = range('a', 'e'); // ['a', 'b', 'c', 'd', 'e']
?>
```

- Die **Kurzschreibweise mit `[]`** ist seit PHP 5.4 verfügbar und wird heute häufiger verwendet als `array()`.
- Der **Spread-Operator `...`** (seit PHP 7.4) ermöglicht das Entpacken von Arrays in andere Arrays.
- Die **`range()`-Funktion** erzeugt Arrays mit aufeinanderfolgenden Werten - praktisch für Schleifen oder Test-Daten.

ÜBUNG: ARRAYS ERZEUGEN

Welche der folgenden sind gültige Möglichkeiten, ein Array in PHP zu erstellen?

- [ ] `$arr1 = [1, 2, 3];`
- [ ] `$arr6 = ['name' => 'Max', 'alter' => 25];`
- [ ] `$arr2 = array(1, 2, 3);`
- [ ] `$arr5 = Array.of(1, 2, 3);`
- [ ] `$arr4 = (1, 2, 3);`
- [ ] `$arr3 = [...$anderes_array];`

FUNKTIONEN

```
function berechneUndPruefe(
        $wert,
        &$istGroesserAlsNull) {
    if ($wert > 0) {
        $istGroesserAlsNull = true;
        return $wert*$wert;
    } else {
        $istGroesserAlsNull = false;
        return 0;
    }
}
```

- Funktionen werden in PHP mit dem Schlüsselwort `function` definiert. Analog zu JavaScript sind weder Parameter- noch Ergebnistyp festzulegen.
- Parameter werden "by value" übergeben, d.h. man kann diese lokal verändern, ohne eine Wirkung beim Aufrufer zu erzeugen.
- Ein-/Ausgabeparameter müssen mit einem vorangestellten `&` gekennzeichnet werden.
- Das Schlüsselwort `return` wird analog zu JavaScript genutzt (mit oder ohne return-Wert).

Zugriff auf globale Variablen und optionale Typangaben

In PHP gelten Variablen innerhalb Ihres jeweiligen Scopes. Damit zum Beispiel eine Variable in einer Funktion genutzt werden kann, kann diese mit dem Schlüsselwort `global` in der Funktion verfügbar gemacht werden.

```php
$something = 42;
function calculate(int $wert): int {
    global $something;
    return $something + $wert;
}
```

- Typangaben bei Parametern und beim Ergebnistyp sind optional; sind sie vorhanden, werden Sie zur Überprüfung **zur Laufzeit** genutzt
- Globale Variable einbinden in der Funktion
- Verwendung von `$something`

ÜBUNG: FUNKTIONEN IN PHP

Welche Aussagen über PHP-Funktionen sind korrekt?

- [ ] Das Schlüsselwort function wird zur Definition verwendet
- [ ] Parameter werden standardmäßig "by reference" übergeben
- [ ] Die Typangabe von Parametern ist optional
- [ ] Funktionen können ohne Rückgabewert definiert werden

ÜBUNG: CALL-BY-REFERENCE

Was passiert, wenn Sie einen Parameter in einer PHP-Funktion ohne `&` übergeben?

```php
function modify($value) { $value = 42; }
$x = 10;
modify($x);
```

- [ ] Der Parameter kann nicht verändert werden
- [ ] Änderungen werden automatisch an die aufrufende Stelle übertragen
- [ ] Änderungen am Parameter bleiben lokal in der Funktion
- [ ] Es kommt zu einem Fehler

ÜBUNG: UNTERSCHIEDLICHE FUNKTIONEN

Analysieren Sie den Code und beantworten Sie die folgende Frage.

```php
function modify1($x) {
    $x = $x * 2;
    return $x;
}
function modify2(&$x) {
    $x = $x * 2;
}

$testValue = 5;

$result = modify1($testValue);
echo "Test-Value (1): $testValue\n";
echo "Funktionsergebnis: $result\n";

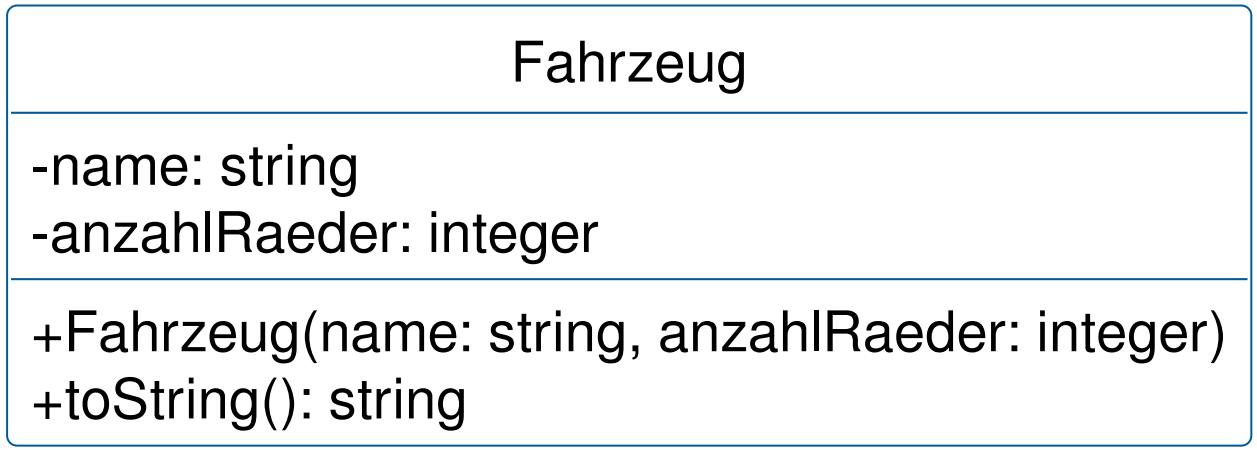
modify2($testValue);
echo "Test-Value (2): $testValue\n";
```

Was wird bei den drei echo-Anweisungen ausgegeben? Erklären Sie warum.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

KLASSEN UND OBJEKTE

Wie man in PHP eine Klasse definiert, veranschaulichen wir anhand einer Fahrzeug-Klasse:





Klassen werden ähnlich zu Java definiert und erzeugt, wobei die Datentyp-Angaben optional sind:

```php
class Fahrzeug {
    private string $name;
    private int $anzahlRaeder;

    public function __construct(string $name, int $anzahlRaeder) {
        $this->name = $name;
        $this->anzahlRaeder = $anzahlRaeder;
    }

    public function __toString(): string {
        return $this->name.", ".$this->anzahlRaeder." Räder";
    }
}
```

- Private Attribute `name` und `anzahlRaeder`
- Konstruktor mit 2 Parametern zur Initialisierung der Attribute über `$this->`. Beachte die Schreibweise mit `__`-Präfix!
- Überschreiben der magischen Funktion `__toString()`. Wird wie in Java aufgerufen, wenn ein Objekt in einen String umgewandelt werden soll.



Klassen können zusätzlich in Namespaces organisiert werden. Dies kann mit Packages verglichen werden und führt dazu, dass der vollqualifizierte Name der Klasse um den Namespace erweitert wird:

```
namespace Model;
class User { ... }
```

Der resultierende Klassenname ist in PHP dann `Model\User`, z.B. kann mittels `new Model\User()` eine neue Instanz erstellt werden.

Mit dem Schlüsselwort `use` lassen sich dann vergleichbar mit Imports in Java die Klassenbezeicher wieder kürzen:

```
use Model\User;
$user = new User();
```

ÜBUNG: KONSTRUKTOREN

Welche Aussagen über Konstruktoren in PHP sind korrekt?

- [ ] Eine Klasse kann mehrere Konstruktoren haben
- [ ] Eine Klasse muss einen Konstruktor haben
- [ ] Der Konstruktor kann Parameter entgegennehmen
- [ ] Der Konstruktor wird mit `__construct()` definiert
- [ ] Der Konstruktor wird automatisch beim Erstellen eines Objekts aufgerufen

ÜBUNG: KLASSEN

Analysieren Sie den Code und beantworten Sie die folgende Frage.

```php
class Person {
    private $name;
    public $age;

    public function __construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
    }

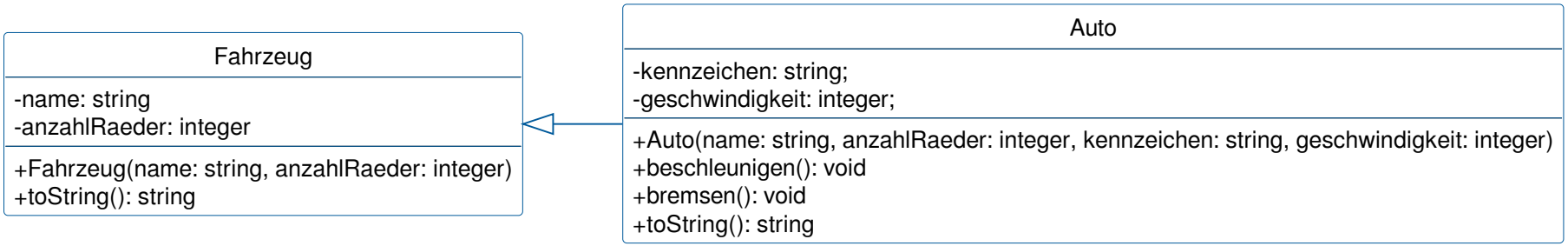
    public function getName() {
        return $this->name;
    }
}
$person1 = new Person("Max", 25);
echo $person1->getName() . "\n";
echo $person1->name . "\n";
echo $person1->age . "\n";
```

Welche der echo-Anweisungen funktionieren und welche erzeugen einen Fehler? Erklären Sie warum.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

VERERBUNG

Zur Veranschaulichung der Nutzung des Vererbungskonzept mit PHP erweitern wird das Fahrzeug-Modell um eine erbende Klasse Auto:



Die Klasse Auto wird in PHP so umgesetzt:

```php
class Auto extends Fahrzeug {
    private $kennzeichen;
    private $geschwindigkeit;

    public function __construct($name, $kennzeichen) {
        parent::__construct($name, 4);
        $this->kennzeichen = $kennzeichen;
        $this->geschwindigkeit = 0;
    }
    public function __toString() {
        return parent::__toString().", Kennzeichen ".$this->kennzeichen
            ", v = ".$this->geschwindigkeit;
    }
    public function beschleunigen() {
        $this->geschwindigkeit += 10; // in km/h
    }
    public function bremsen() {
        $this->geschwindigkeit -= 10; // in km/h
        if ($this->geschwindigkeit < 0)
```

- Klasse Auto erbt von Fahzeug.
- Aufruf des Oberklassen-Konstruktors
- Zugriff auf überschriebene Methode

6.3 FORMULARVERARBEITUNG



Browser und PHP-Skript kommunizieren über HTTP-Anfragen und -Antworten

Mit der Anfrage werden Daten geschickt, die verarbeitet werden sollen und resultieren in einem generierten Ergebnis, z.B. in HTML.

Mittels HTML lassen sich Formulare umsetzen, mit denen die Eingabe von Daten möglich ist. Im Abschnitt über HTML-Formulare wurde erklärt, wie diese gestaltet werden und das man meist Logik auf dem Webserver zur Verarbeitung der Formulardaten benötigt.



Mittels PHP sind wir in der Lage diese Verarbeitung zu realisieren. Anhand eines Beispiels wird erklärt, wie ein Formular mit PHP serverseitig verarbeitet werden kann.

Daten werden entweder in der URI als Query transportiert oder im Payload eines POST-HTTP-Request

VERARBEITUNG VON GET-METHODEN

Folgende Animation zeigt, wie ein Formular geladen wird und die Eingaben anschließend verarbeitet werden



Erklärung zur Animation (1)

- Mit dem Laden der URL wird der PHP-Code ausgeführt, da keine Daten übermittelt wurden, sind keine Ausgaben sichtbar (ggf. kommt es zu Warnings)
- Gibt der Nutzer Daten ein (Schritt 2) und drückt auf absenden, werden die Daten (wegen method="get") in die URI eingebettet
- Dabei wird die Seite erneut vom Webserver geladen
- Die PHP-Generierung erzeugt dieses mal eine Ausgabe mit den übermittelten Daten

VERARBEITUNG VON POST-METHODEN

Folgende Animation zeigt, wie ein Formular geladen wird und die Eingaben anschließend verarbeitet werden

Erklärung zur Animation (2)

- Wie auch bei der 1. Animation beginnt der Ablauf mit dem Laden der URL aber ohne Ausgaben
- Gibt der Nutzer Daten ein (Schritt 2) und drückt auf absenden, werden die Daten (wegen method="post") in als Payload in die Anfrage eingebettet
- Dabei wird die Seite erneut vom Webserver geladen
- Die PHP-Generierung erzeugt dieses mal eine Ausgabe mit den übermittelten Daten

METHODEN UND VARIABLEN

Wichtige Variablen für den Zugriff auf abgesendete Formulardaten:

- `$_GET`: Ist ein Array, welches als Index die Namen der Formularfelder und deren Eingaben als Wert enthält (vgl. `<input name="test">` wäre `$_GET["test"]`), wenn das Formular mittels `method="get"` abgesendet wurde **oder** direkt an die URI ein Query-Teil ergänzt wird, vgl. `localhost/test.php?test=42`
- `$_POST`: Ist ein Array, welches als Index die Namen der Formularfelder und deren Eingaben als Wert enthält (vgl. `<input name="test">` wäre `$_POST["test"]`), wenn das Formular mittels `method="post"` abgesendet wurde



Alternativ gibt es `$_REQUEST`, dies ist ebenfalls ein Array, welches als Index die Namen der Formularfelder und deren Eingaben als Wert enthält (vgl. `<input name="test">` wäre `$_GET["test"]`).

Hierbei spielt es keine Rolle, ob das Formular mittels `method="post"` oder `method="get"` abgesendet wurde. Gibt es sowohl einen Query-Parameter und einen Parameter im Payload, der den selben Namen enthält kommt es zu Überschneidungen (ein Wert ist dann nicht sichtbar).

Wichtige Methoden für die Überprüfung

- `isset(mixed):boolean`: Prüft, ob ein Wert gesetzt ist, z.B. `isset($_GET['test'])`, liefert true, wenn ein Wert gesetzt wurde, auch wenn dieser leer ist
- `empty(mixed):boolean`: Prüft, ob ein Wert leer ist, z.B. `empty($_GET['test'])`, liefert true, wenn ein Wert nicht leer ist
- `strlen(string):int`: Gibt die Länge einer Zeichenkette zurück
- `sizeof(array):int`: Gibt die Länge eines Arrays zurück

Alternativ der Null Coalescing Operator:

```
$name = $_GET['name'] ?? 'anonymous';
```

FORMULAR WURDE ABGESENDET

Wie erkennen Sie, dass ein Formular geladen wird oder abgesendet wurde?

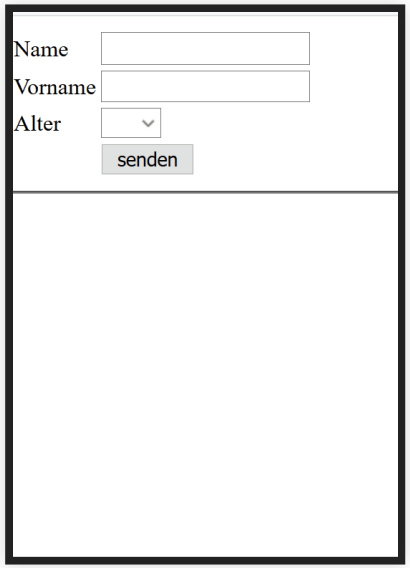
Prüfen Sie, ob mit der aktuellen PHP-Skript Ausführung Informationen in `$_GET` oder `$_POST` enthalten sind. Dies kann auf unterschiedliche Art und Weise passieren. Es bietet sich zum Beispiel eine `action`-Information an, welcher über einen Button gesetzt wird:

```
<form>
    <button name="action" value="a" type="submit">A</button>
    <button name="action" value="b" type="submit">B</button>
    <?php
    if(isset($_GET['action']) && $_GET['action'] == 'a') {
        echo "Aktion a";
    } else if(isset($_GET['action']) && $_GET['action'] == 'b') {
        echo "Aktion b";
    } else {
        echo "keine Aktion";
    }
    ?>
</form>
```

Dies lässt sich ebenfalls mittels Links erreichen

```
<p><a href="?action=a">A</a></p>
<p><a href="?action=b">B</a></p>
<?php
if(isset($_GET['action']) && $_GET['action'] == 'a') {
    echo "Aktion a";
} else if(isset($_GET['action']) && $_GET['action'] == 'b') {
    echo "Aktion b";
} else {
    echo "keine Aktion";
}
?>
```

BEISPIEL EINES ERWEITERTEN FORMULARS



- Das Formular wird geladen in dem die Webseite geöffnet wird.
- Die eingegebenen Formulardaten (Name, Vorname und Alter) werden durch Klick auf den Senden-Button ("Submit") zum Server gesendet.
- Das gleiche PHP-Skript, welches das Formular angezeigt hat, erkennt nun, dass Daten vorliegen und generiert eine Ausgabe der Werte.

Erläuterung des Beispiel-Codes: das Formular

```
<form action="Formular-Beispiel.php" method="post">
    <table>
        <tr><td>Name</td><td><input type="text" name="name"></td></tr>
        <tr><td>Vorname</td><td><input type="text" name="vorname"></td></tr>
        <tr>
            <td>Alter</td>
            <td>
                <select name="alter">
                    <option value="0"> </option>
<?php
                    for ($i = 18; $i <= 99; ++$i) {
                        echo "<option value='$i'>$i</option>\n";
                    }
?>
                </select>
            </td>
        </tr>
        <tr><td> </td><td><input type="submit" value="senden"></td></tr
    </table></form>
```

- Beim "Submit" wird das Skript "Formular-Beispiel.php" mit POST-Werten aufgerufen, um die Folgeseite zu generieren.
- Anhand der name-Attribute kann das PHP-Skript die Werte für Name, Vorname und Alter lesen.
- Die Alter-Combobox wird über eine Schleife befüllt.
- Nach Klick auf den Submit-Button sendet der Browser die Formulardaten an den Server.

Erläuterung des Beispiel-Codes: Verarbeitung der Daten



```
<hr>
<?php
if (isset($_POST['name'])) {
    $name = $_POST['name'];
    $vorname = $_POST['vorname'];
    $alter = $_POST['alter'];
?>

    <h1>Werte:</h1>
    <table>
        <tr><td>Name</td><td><?php echo $name; ?></td></tr>
        <tr><td>Vorname</td><td><?php echo $vorname; ?></td></tr>
        <tr><td>Alter</td><td><?php echo $alter; ?></td></tr>
    </table>
<?php
} // end if
?>
```

- Zuerst wird mit `isset($_POST['name'])` geprüft, ob für den Namen "name" ein Wert vorliegt. Daran ist zu erkennen, ob das Formular ausgefüllt zum Server gesendet wurde.
- Die Formulardaten befinden sich im vorbelegten `$_POST`-Array (vgl. Method-Parameter des Formulars!).
- Die Daten werden in Variable kopiert...
- ... und unten ausgegeben.

Weitere Anmerkungen:

- Wenn im Browser ein Formular submitted wird und die Formular-Methode "POST" ist, so werden die Werte der Formularfelder (Text, Kombobox, Checkbox etc.) zum Server gesendet und dort vor der Ausführung des PHP-Skripts in dem Array $_POST gespeichert.
- $_POST ist ein vordefiniertes Array. Als sog. "Superglobal" muss es nicht mit dem Schlüsselwort "global" deklariert werden.
- Wird ein Formular mit der Methode "GET" submitted, so können die Formularwerte analog im Array $_GET gelesen werden.

Achtung: mit "GET" übertragene Inputs bewirken ein sogenanntes URI-Rewriting, da der Query-String mit den Werten einfach an die URL angehängt wird!

Beispiel: Gibt man Maier, Anna, 25 ein und sendet das Formular, so lautet der letzte Teil des im Browser angezeigten URI:

```
Formular-Beispiel.php?name=Maier&vorname=Anna&alter=25
```

Bei vertraulichen Daten sollte daher stets mit POST und Verschlüsselung (HTTPS) gearbeitet werden.

ÜBUNG: POST-DATEN

Wie greift man in PHP auf Daten zu, die mit HTTP-POST bzw. einem HTML-Formular mit `method="post"` versendet werden?

- ☐ Durch das Array `$data['post']`
- ☐ Durch die Funktion `getPost()`
- ☐ Durch die Variable `$post`
- ☐ Durch das superglobale Array `$_POST`

ÜBUNG: GET-DATEN

Welche der folgenden Aussagen über die GET-Methode sind korrekt?

- [ ] Die übertragenen Daten können gebookmarkt werden
- [ ] Die Methode ist ideal für Formulare zur Authentifizierung
- [ ] Die Methode ist ideal für Suchformulare
- [ ] Die Methode ist sicher für sensible Daten
- [ ] Die Daten sind in der URL sichtbar

ÜBUNG: GIBT ES DIE DATEN?

Welche Aussage über `isset($_POST['name'])` ist korrekt?

- [ ] Prüft, ob der Wert von `$_POST['name']` null ist
- [ ] Prüft, ob das POST-Array leer ist
- [ ] Prüft, ob der Index `name` im POST-Array existiert
- [ ] Prüft, ob der Wert von `$_POST['name']` leer ist

ÜBUNG: NULL COALESCING OPERATORS

Was ist die korrekte Verwendung des Null Coalescing Operators für ein Formularfeld?

- [ ] `$plz = $_POST['plz'] || "12345";`
- [ ] `$street = isset($_POST['street']) ? $_POST['street'] : "Esplanade";`
- [ ] `$city = $_POST['city'] ?? "Ingolstadt";`
- [ ] `$name = $_POST['name'] ? $_POST['name'] : "Anonym";`

ÜBUNG: FORMULAR UND VERARBEITUNG

Ein PHP-Skript enthält sowohl ein HTML-Formular als auch dessen Verarbeitung, welche der nachfolgenden Aussage ist richtig?

```php
<?php
if(isset($_POST['submit'])) {

echo "Verarbeitung...";
}
?>
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
    <input type="text" name="data">
    <button type="submit" name="submit">Senden</button>
</form>
```

Welche Aussage ist richtig?

- [ ] PHP führt nur den Verarbeitungsteil aus und überspringt die Formular-Generierung
- [ ] PHP führt entweder nur den Verarbeitungsteil oder nur die Formular-Generierung aus
- [ ] PHP führt das gesamte Skript aus und generiert sowohl die Verarbeitung als auch das Formular
- [ ] PHP führt je nach Platzierung nur den Code vor oder nach dem Formular aus

ÜBUNG: FORMULARVERARBEITUNG

Gegeben ist folgendes HTML-Formular (`check.php`) und drei PHP-Block-Positionen (A, B und C).

```
<?php /* ... A ... */ ?>
<form action="check.php" method="post">
    <label for="ageField">Alter eingeben:</label>
    <input type="number" id="ageField" name="age" min="0" max="150">
    <button type="submit" name="btn" value="check">Prüfen</button>
    <?php /* ... B ... */ ?>
</form>
<?php /* ... C ... */ ?>
```

Beschreiben Sie, wie dieses Formular verarbeitet werden könnte, wenn es abgesendet wurde, und an welcher Stelle Sie die Verarbeitung platzieren würden (A, B oder C).

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

FORMULARE WIEDER AUSFÜLLEN

Wird die Webseite z.B. über den Submit-Button neu geladen, wird das Formular in seiner Start-Konfiguration angezeigt

Sollen die Eingaben des abgesendeten Formulars erneut angezeigt werden, ist es notwendig, diese in HTML an die richtigen Stellen zu generieren. Für eine HTML-Formularfeld bedeutet dies, dass das Value-Attribut zu füllen ist.

```
<form>
  <input name="test" value="<?= $_GET['test'] ?>"><button>Send</button>
</form>
```

Je nach Typ des Formularfeldes, ist es wichtig das HTML-Dokument korrekt zu generieren

- Checkbox / Radio: benötigt ein `checked`, damit dieses wieder ausgewählt wird
- Textarea: benötigt die Ausgabe zwischen Start- und End-Tag
- Select: benötigt `selected` an der Option, die ausgewählt werden soll

Beispiel mit Radio-Button

```
<form>
  <input name="t" value="1" <?= $_GET['t'] == '1' ? 'checked' : '' ?>>
  <input name="t" value="2" <?= $_GET['t'] == '2' ? 'checked' : '' ?>>
  <button>Send</button>
</form>
```

Beispiel mit Textarea

```
<form>
  <textarea name="test"><= $_GET['test'] ?></textarea>
  <button>Send</button>
</form>
```

Beispiel mit Select

```
<form>
  <select name="test">
    <option <?= $_GET['t'] == '1' ? 'selected' : '' ?>>1</option>
    <option <?= $_GET['t'] == '2' ? 'selected' : '' ?>>2</option>
  </select>
  <button>Send</button>
</form>
```

ÜBUNG: WICHTIGE HTML-ATTRIBUTE ZUR GENERIERUNG

Welche Aussagen zu Formularfeldern und deren Werten sind korrekt?

- [ ] Bei Checkboxen wird der Status über `value='checked'` gesteuert
- [ ] Bei einer Textarea wird der Wert zwischen öffnendem und schließendem Tag platziert
- [ ] Bei Select-Elementen wird die ausgewählte Option über das selected-Attribut gesteuert
- [ ] Checkboxen werden über das checked-Attribut (ohne Wertzuweisung) als ausgewählt markiert
- [ ] Textareas verwenden das value-Attribut

ÜBUNG: FORMULARE KORREKT GENERIEREN

Gegeben ist folgender HTML/PHP-Code für ein Newsletter-Anmeldeformular:

```
<form action="subscribe.php" method="post">
    <label for="email">E-Mail:</label>
    <input type="email" id="email" name="email" <?php /* CODE A */ ?>>

    <label for="consent">Ich stimme dem Erhalt des Newsletters zu</label>
    <input type="checkbox" id="consent" name="consent" <?php /* CODE B */ ?>

    <button type="submit" name="submit">Anmelden</button>
</form>
```

Was muss an den Stellen CODE A und CODE B gemacht werden, sodass das die bereits eingegebenen Werte wieder im Formular erscheinen, nachdem das Formular abgesendet wurde.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

6.4 SESSION-HANDLING

- Webserver haben typischerweise keine Information über vorangegangene Interaktionen mit einem Client.
- Der Grund hierfür: HTTP ist zustandslos, was u.a. eine gute Skalierbarkeit ermöglicht!
- Ist ein Webserver überlastet, so kann ein anderer mit neuen Anfragen desselben Browsers betraut werden. Dies ist einfach möglich, da der Webserver alle zur Bearbeitung der Anfrage nötigen Informationen in der HTTP-Request-Nachricht erhält.
- Viele Webanwendungen benötigen jedoch ein "Gedächtnis", d.h. auf Serverseite müssen bestimmte Daten bzgl. eines Nutzers bekannt sein.
- Ohne diese Daten müsste sich ein Benutzer bei jedem Seitenaufruf erneut authentifizieren oder man müsste bestellte Artikel jedes Mal neu auswählen (Warenkorb!).
- Weitere Beispiele sind Personalisierung von Webseiten, das Speichern von Interessen des Benutzers oder etwa gezielte Werbung.

Wie kann man das "Gedächtnis" realisieren?

- Der Server verwaltet pro Client eine sogenannte Session.
- Eine Session ist eine Folge von Kommunikationsvorgängen, die sich auf einen gemeinsamen Zustand beziehen.
- In einem Webshop werden beispielsweise die Anmeldung des Benutzers, der virtuelle Einkaufswagen, die Artikel-Historie etc. als Session-Zustand gespeichert.
- Wichtig ist, dass die Benutzersessions voneinander getrennt verwaltet werden, so dass z.B. ausgewählte Produkte dem Einkaufswagen des jeweiligen Benutzers zugeordnet werden.

Wie können Sessions implementiert werden?

In der Praxis werden **hidden fields**, **URI-Rewriting** sowie **HTTP Cookies (bzw. LocalStorage)** eingesetzt, nicht selten in einer Kombination aus mehreren der genannten Techniken. Die drei Techniken werden nun erklärt.



Das folgende PHP-Projekt enthält drei Skripte zur Demonstration der drei Möglichkeiten der Session-Realisierung.

- `hidden-input.php`: der Einkaufswagen wird in einem "hidden input" transportiert.
- `uri-rewriting.php`: der Einkaufswagen wird als "Query String" im URI transportiert.
- `cookie.php`: der Einkaufswagen wird in einem Browser-Cookie gespeichert.

Nutzen Sie das bereitgestellte Projekt-Archiv session-examples.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

SESSION-REALISIERUNG MIT HIDDEN FIELDS

Grundidee:

- Der Server schreibt Zustandsinformation in ein verstecktes Formular-Feld ("hidden field") und transportiert die Zustandsinformation auf diese Weise zum Browser.
- Wird das Formular zurückgesendet (submit), so werden die versteckten Informationen automatisch wieder zum Server gesendet und dann von diesem ausgewertet, um die nächste Aktion zu bestimmen und die benötigten Zustandsdaten zu extrahieren.
- Hier ein Beispiel aus der Adressverwaltung

```
<form method="post" action="index.php">
    <input type="hidden" id="aktion" name="aktion" value="suchen">
</form>
```

Erläuterung des Beispiels:

```
<form method="post" action="index.php">
    <input type="hidden" id="aktion" name="aktion" value="suchen">
</form>
```

- Das Formular wird nach dem Submit stets an das Skript "index.php" gesendet.
- Der daraufhin aufgerufene Controller entscheidet anhand des hidden inputs, welche Aktion als nächste auszuführen ist.
- Im Beispiel ist dies die Aktion "suchen". Entsprechend werden die zum Filter passenden Adressen ermittelt und angezeigt.

Die Art und Weise, wie hidden fields zur Verwaltung der Zustandsinformation eingesetzt werden, ist vielfältig:

- Beispielsweise könnte man eine **SessionID** in einem hidden field speichern. Auf Serverseite kann dann mit der **SessionID** ein Variablenpool für die Session geöffnet werden, um auf alle relevanten Informationen zugreifen zu können.
- In ASP.NET wird ein sog. ViewState als verschlüsselter Wert in einem hidden field transportiert. Zurück auf der Serverseite wird der ViewState entschlüsselt und dann zur Wiederherstellung des Session-Zustands genutzt.
    - Der Vorteil dieser Methode ist, dass der Server keinerlei Information zu einer Session vorhalten muss (stateless server).
    - Nachteilig ist unter Umständen die Größe der mit jedem Aufruf hin und wieder zurück zu sendenden Daten.

Generell besitzt die "hidden field"-Methode folgende **Vorteile**:

- sie ist sehr einfach zu realisieren und funktioniert mit jedem Browser.
- Die Session-Information ist von außen nicht sichtbar.
- Bookmarks bzw. die Weitergabe der URI ist ungefährlich.

Nachteilig ist, dass normale Links nicht genutzt werden können, da die Inputs nur beim Submit eines Formulars gesendet werden.

ÜBUNG: HIDDEN-FIELDS

Welche Aussagen über versteckte Formularfelder sind richtig?

- [ ] Sie sind im Quellcode (HTML) der Seite für Benutzer nicht sichtbar
- [ ] Sie bleiben auch nach Schließen des Browsers erhalten
- [ ] Sie werden im HTML-Code in einem Input-Tag mit type="hidden" definiert
- [ ] Die Daten müssen bei jedem Formularaufruf erneut übertragen werden

URI REWRITING

URI-Rewriting ist eine Technik, bei der der Haupt-URL einer Webanwendung (z.B. www.example.com) um Session-Informationen ergänzt wird:

- Beim Start einer Session (z.B. beim ersten Aufruf von www.example.com) wird eine Session-ID erzeugt.
- Jeder Seite, die zur Session gehört, wird anschließend eine Session-ID in den URI eingefügt.
- Beispiel: http://www.example.com?sessionID=123456789
- Erhält der Server dann eine Anfrage, kann er anhand der Session-ID den zur Session gehörenden Variablen-Pool aktivieren (typischerweise aus einer Datei).
- Dieser enthält dann Informationen wie Login-Status (eingeloggt oder nicht), Warenkorb-Inhalt usf.

Versteckte Formularfelder und URI-Rewriting werden typischerweise in einer Session je nach Art des Server-Aufrufs verwendet:

- Bei normalen Links (<a>-Tag) wird URI-Rewriting genutzt.
- Bei Formularen wird ein hidden field mit der Session-ID genutzt.

Die **Vorteile** von URI-Rewriting sind, dass es

- einfach zu realisieren ist und
- unabhängig von den Browsereinstellungen immer korrekt funktioniert.

Nachteile können sein, dass

- Session-Information in der Adresszeile öffentlich sichtbar sind und
- versehentlich "veröffentlicht" werden können, z.B. durch Bookmark'en und Weitergabe des Links oder versehentliches Anhängen der Information an externe Links.

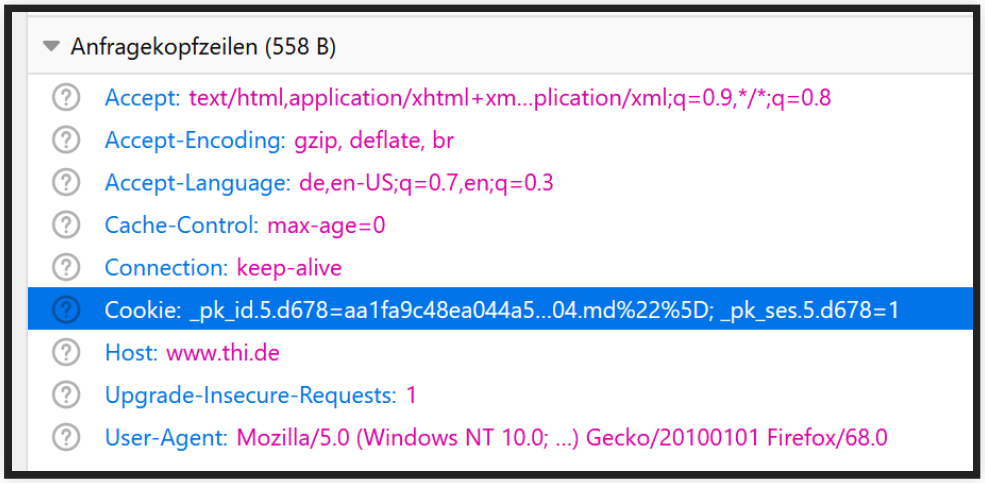
ÜBUNG: URL-PARAMETER

Ein Benutzer speichert folgende URL als Lesezeichen: `shop.php?warenkorb=42`*.*
Was passiert beim erneuten Aufrufen des Lesezeichens nach einer Woche?

- ☐ Es wird automatisch immer ein neuer Warenkorb erstellt
- ☐ Der aktuelle Warenkorbinhalt des Benutzers wird angezeigt, da der Warenkorb in der URL gespeichert ist
- ☐ Der Warenkorb mit ID=42 wird vom Server angefragt, sofern dieser noch in der Datenbank gespeichert ist
- ☐ Die Seite wird immer ohne Warenkorb aufgerufen

HTTP COOKIES

- Cookies sind eine von der Firma Netscape entwickelte Erweiterung des HTTP-Protokolls.
- Der Server speichert Informationen auf dem Client in Form von Schlüssel-Wert-Paaren als ASCII-Text.
- Die Cookie-Informationen können sowohl im Request-, also auch im Reply-Header übertragen werden.
- thi.de setzt beispielsweise Browser-seitig folgende Cookies und sendet diese mit jeder Anfrage zurück an den Server:



Cookie-Attribute

Attribut	Beschreibung
name=<value>	eigentlicher Inhalt als Schlüssel-Wert-Paar
expires=<datum>	Nach dem Datum wird das Cookie nicht mehr gesendet
domain=<domain>	Bestimmt, zu welchen Domains das Cookie geschickt wird, Default ist der Rechnername, von dem das Cookie kommt
path=<pfad>	definiert eine Untermenge der URIs in der Domäne, für die das Cookie gültig ist. Default ist der Pfad der Ressource, die das Cookie gesetzt hat
secure	wenn vorhanden, wird das Cookie nur bei einer HTTPS-Verbindung übertragen

Begrenzungen und Möglichkeiten:

- Der Browser speichert maximal 300 Cookies und
- max. 4KB an Daten pro Cookie
- Je Server bzw. Domain sind max. 20 Cookies erlaubt.
- Ein Cookie kann gelöscht werden durch Setzen eines abgelaufenen expires-Wertes (Client- oder Server-seitig) – das abgelaufene Cookie wird dann vom Browser selbst gelöscht

Ablauf der Cookie-Übertragung:

- Vor jedem Request ermittelt der Browser die zur URI passenden Cookies.
- Er wendet dabei SOP (Same Origin Policy) als Sicherheitskonzept an, d.h. es werden nur die Cookies übertragen, für die
    - die Domain
    - das Verzeichnis und
    - das Protokoll passen.
- Die zu übertragenden Cookies werden im HTTP Request Header vom Client an den Server mitgeschickt.

Beispiel um ein Cookie in PHP zu setzen

Die PHP-Funktion `setcookie($name, $wert, $config)` wird verwendet, um ein neues Cookie zu setzen. Dies platziert in die HTTP-Antwort vom Server die notwendige Information im Header.

```php
setcookie(
    'my-cookie',
    '123',
    [
        'expires' => time() + 3600, // Gültig 1 Stunde
        'path'     => '/',          // Gültig für gesamte Site
    ]
);
```

Beispiel um ein Cookie in PHP zu lesen

Wird ein HTTP-Request mit Cookies gesendet, können diese über ihren Namen in dem globalen Array `$_COOKIE` ausgelesen werden.

```php
if(isset($_COOKIE['my-cookie'])) {
    echo "Wert: " . $_COOKIE['my-cookie'];
}
```

Beispiel um ein Cookie in PHP zu löschen

Damit ein Cookie im Browser gelöscht wird, müssen wir ein Cookie setzen und eine "expire" Zeit setzen, die in der Vergangenheit liegt.

```php
setcookie(
    'session',
    '',
    [
        'expires' => time() - 3600, // nicht mehr gültig
        'path'     => '/',          // Gültig für gesamte Site
    ]
);
```

Vorteile von Cookies:

- Cookies werden durch den Browser verwaltet:
- Der Benutzer kann Cookies ablehnen bzw. diese jederzeit selbst löschen.
- Durch SOP ist sichergestellt, dass Cookies nicht an unberechtigte Webseiten gesendet werden.
- Die lokale Speicherung gewährleistet einen gewissen Schutz der Cookies vor fremdem Zugriff.
- Für Anbieter von Websites ergibt sich, dass Informationenlangfristig und (weitgehend) unsichtbar für den Nutzer gespeichert werden können.

Nachteile/Probleme von/mit Cookies:

- Cookies können per Cross-Site-Scripting (XSS) gelesen werden, sind also nicht 100% sicher.
- Zudem werden Cookies zur Nachverfolgung des Benutzer-Surfverhaltens genutzt (daher die typische Cookie-Frage auf der Startseite).
- Abhilfe: Im Browser können Cookies gelöscht werden, zudem kann man die Domänen festlegen, die Cookies setzen dürfen.

ÜBUNG: HTTP-COOKIES

Welche Eigenschaften haben Cookies?

- [ ] Sie können beliebig große Datenmengen speichern
- [ ] Sie können vom Server über den HTTP-Response im Browser gespeichert werden
- [ ] Sie werden bei HTTP-Anfragen an die entsprechende Domain automatisch im Header mitgesendet
- [ ] Sie werden nur auf dem Server gespeichert
- [ ] Sie werden im Browser des Benutzers gespeichert

ÜBUNG: NOTWENDIGKEIT VON SITZUNGSMANAGEMENT

Erklären Sie mit eigenen Worten, warum Konzepte zum Sitzungsmanagement bei der Arbeit mit PHP und HTTP notwendig ist.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

PHP SESSIONS

PHP bietet ein eigenes Session-Konzept an:

- Serverseitig wird hierzu ein Array namens `$_SESSION` bereitgestellt, welches alle Variablen der Session eines Clients enthält.
- Die Session wird durch Aufruf der Methode `session_start()` initialisiert bzw. aus einer Datei wieder hergestellt.
- Jede Session hat eine eindeutige Session ID, anhand derer die Daten aus der internen Datei geladen werden. Die Session ID wird jeder Reply-Nachricht als Cookie mitgegeben und wird somit im Browser für künftige Anfragen gespeichert.
- Nach Beendigung eines Requests speichert PHP automatisch das `$_SESSION`-Array in einer internen Datei.
- Wird die Session eine gewisse Zeit (session timeout) lang nicht genutzt, so wird sie automatisch gelöscht.

Beispiel zum Setzen von Sitzungsdaten

Initial muss die Sitzung gestartet werden, anschließend können Daten in `$_SESSION` gespeichert werden.

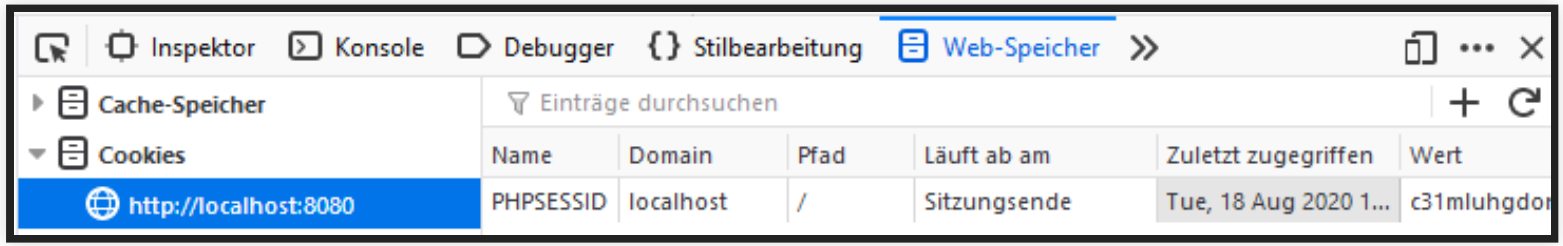
```
session_start();
$_SESSION['my-session-var'] = "Beispiel";
```

Sind Informationen gespeichert, können diese jederzeit verwendet werden. Zum Beispiel um diese auszugeben:

```
<p><?= echo $_SESSION['my-session-var'] ?></p>
```

Wie kann eine Session Browser-seitig gelöscht werden?

Das Cookie mit der Session ID muss gelöscht werden, dann beginnt der Server beim nächsten Aufruf eine neue Session.



Sitzungsdaten in PHP löschen

Wird der Cookie im Client entfernt, bedeutet dies nicht, dass die Daten im Server ebenfalls gelöscht wurden, die für die Sitzungskennung gespeichert wurden.

```php
session_start();

unset($_SESSION['my-session-var']);
```

ÜBUNG: SESSION-IDS IN HTTP-ANTWORTEN

Wie wird die Session-ID (PHPSESSID) einer PHP Session standardmäßig an den Client übermittelt?

- [ ] Als Cookie
- [ ] Als eigener HTTP-Header (nicht Cookie)
- [ ] Als URL-Parameter (URL-Rewriting)
- [ ] Als verstecktes Formularfeld

ÜBUNG: SESSIONDATEN IN PHP

Welche superglobale Variable wird in PHP für den Zugriff auf Session-Daten verwendet?

- [ ] $_COOKIE
- [ ] $GLOBALS
- [ ] $_SESSION
- [ ] $_SERVER

ÜBUNG: SESSION MECHANISMUS IN PHP

Erklären Sie, was die Funktion `session_start()` macht. Warum muss diese aufgerufen werden?

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

ÜBUNG: KEIN AUFRUF DER START-METHODE

Welche Konsequenzen hat es, wenn `session_start()` nicht aufgerufen wird?

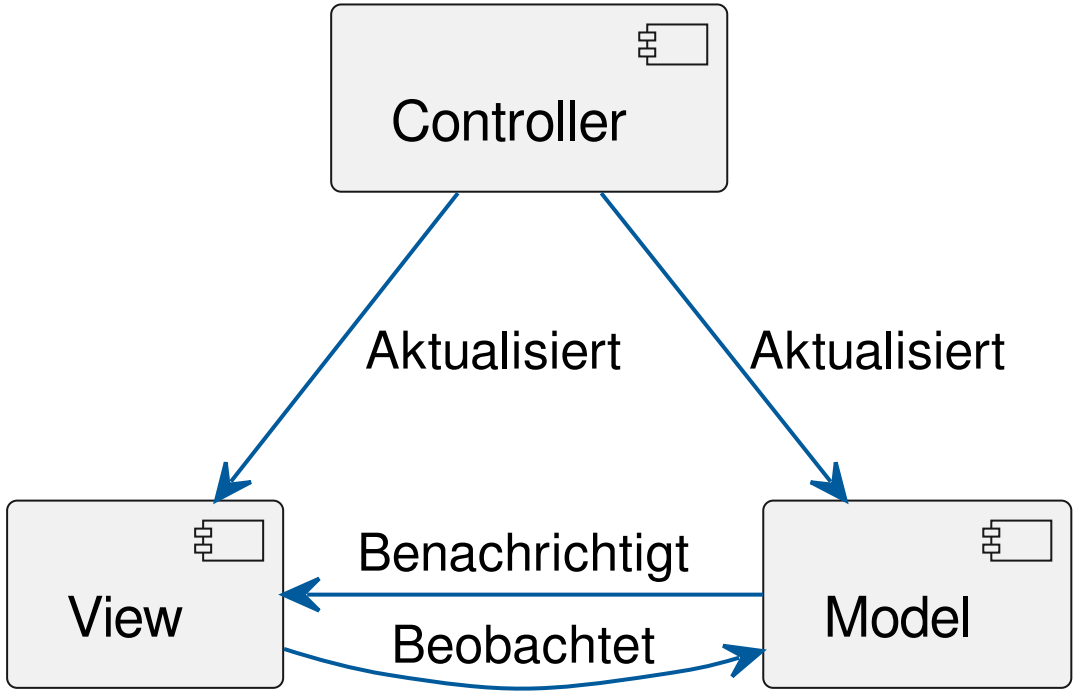
Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

6.5 DAS MODEL-VIEW-CONTROLLER MUSTER

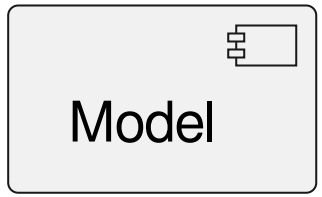
Motivation

- Bei kleineren Anwendungen werden die Aspekte Datenein-/ausgabe (View), Steuerung der Anwendung (z.B. Reaktion auf Ereignisse, Controller) und Datenverarbeitung -und -speicherung (Model) häufig zusammen implementiert.
- Bei größeren Anwendungen bedingt diese Vorgehensweise typischerweise Wartungsprobleme bis hin zur Unwartbarkeit.
- Ein Lösungsansatz für obiges Problem ist das Model-View-Controller Muster. Es trennt View, Controller und Model voneinander und minimiert dadurch die Aufwände für Erweiterung und Fehlersuche.

Trennung von View, Controller und Model

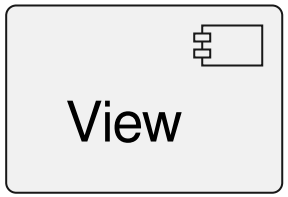


Model



- Zentrale Komponente für die Datenhaltung
- Enthält die Geschäftslogik und Datenverarbeitung
- Arbeitet unabhängig von der visuellen Darstellung
- Beispiel: Datenbankzugriffe, Berechnungen, Validierungen

View



- Zuständig für die Darstellung der Daten
- Präsentiert die Informationen aus dem Model
- Leitet Benutzerinteraktionen an den Controller weiter
- Beispiel: HTML-Seiten, UI-Komponenten, Formulare

Controller



- Vermittler zwischen Model und View
- Verarbeitet Benutzereingaben und Aktionen
- Steuert den Datenfluss zum Model und die Aktualisierung der View
- Beispiel: Formularverarbeitung, Routing, Anwendungslogik

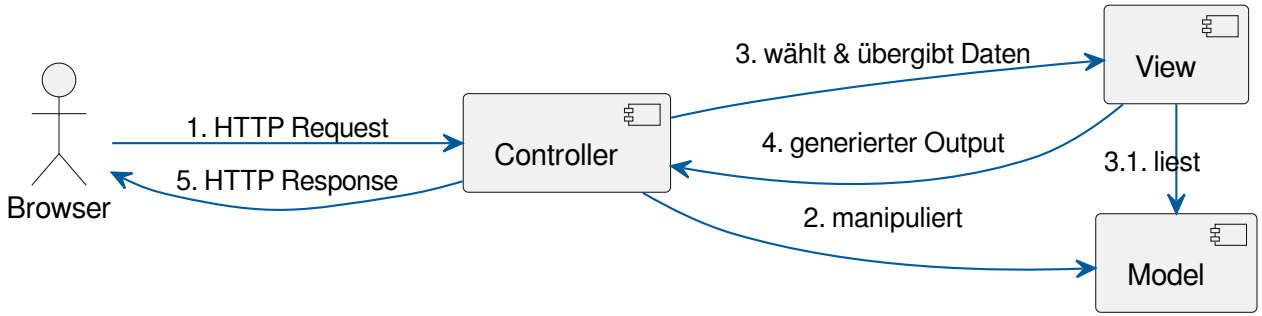
Die Trennung dieser drei Komponenten ermöglicht:

- Bessere Wartbarkeit durch klare Zuständigkeiten
- Einfachere Testbarkeit der einzelnen Komponenten
- Flexibilität bei Änderungen (z.B. neue Views hinzufügen)



Für Webanwendungen kann die angepasste MVC2-Variante verwendet werden, welche im folgenden erklärt wird

DAS MVC2-MUSTER



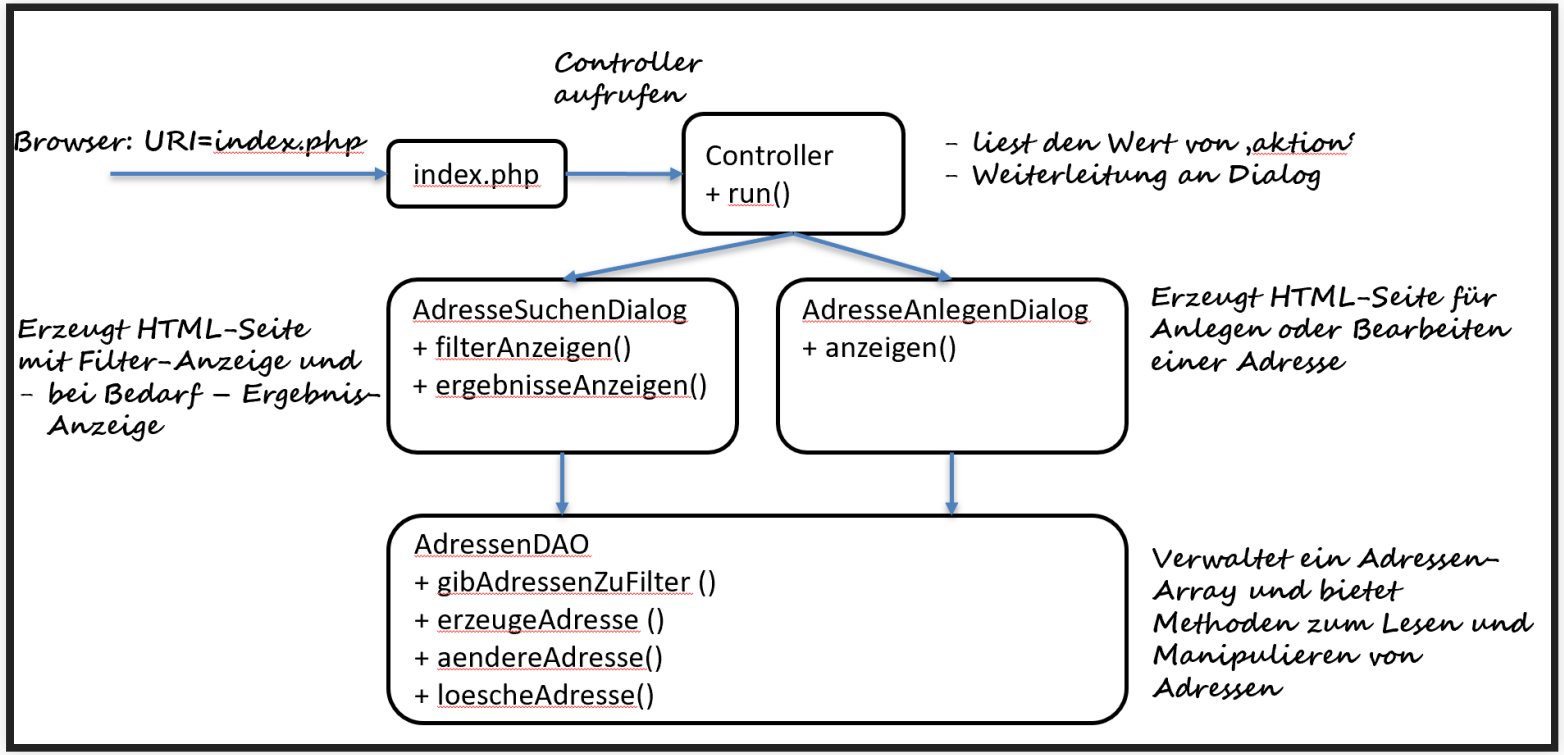
- Eingehende Anfragen der Browser werden vom Controller empfangen.
- Der Controller ermittelt den aktuellen Status des aufrufenden Clients und bestimmt die nächste Aktion.
- Anschließend schreibt der Controller ggf. empfangene Daten ins Model zurück und besorgt sich die für die nächste Aktion benötigten Daten.
- Die View wird nun mit den entsprechenden Daten aufgerufen, um die Folgeseite zu erzeugen.
- Die erzeugte Seite wird an den Aufrufer gesendet.

MVC2 MIT PHP AM BEISPIEL DER ADRESSVERWALTUNG

Im folgenden Beispiel befindet sich ein fragmentarisches Beispiel für eine mögliche Umsetzung einer einfachen Adressverwaltung. Das Beispiel implementiert eine zentrale Dialogsteuerung (Controller-Klasse) gemäß dem MVC2-Muster.

Nutzen Sie das bereitgestellte Projekt-Archiv adress-management.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Die Grundarchitektur ist im Folgenden dargestellt:



Erläuterungen:

- index.php: Um zu gewährleisten, dass der Controller stets die Kontrolle erhält, um die als nächstes anzuzeigende Seite zu ermitteln, führt jeder Server-Aufruf (Formular-Submit) zu index.php.
- Dialogsteuerung (Control):
  - Jede HTML-Seite erhält einen hidden-input (type="hidden") mit dem Namen "aktion".
  - Dieser Wert wird zum Server transportiert und dort ausgewertet, um den nächsten Dialog zu generieren.
  - Client-seitig wird dieser Input durch JS-Funktionen (je nach ausgelöstem Ereignis) entsprechend gesetzt.
- Datenhaltung (Model):
  - Ein Adresse wird durch eine Instanz der Klasse "Adresse" realisiert.
  - Die Adressdaten werden in einem Data Access Object (DAO) gespeichert und sind über die DAO-API lesbar bzw. veränderbar.

- Dialoge (View):
  - Die einzelnen Seiten werden durch Instanzen der Klassen "*Dialog" erzeugt.
- Verwendung von PHP-Sessions:
  - Zur Wiederherstellung des DAOs (Adressenliste!) und des Controller-Objekts zwischen den einzelnen Serveraufrufen wird eine PHP-Session verwendet.
  - Eine Session ist ein Variablenpool, welcher durch "session_start()" aus einer Datei wiederhergestellt werden kann und bei Skript-Ende automatisch gespeichert wird.
  - Die Variablen sind über das globale Array $_SESSION zugänglich.
  - In der Datei "index.php" ist die Wiederherstellung der Session implementiert.

AUTOLOADING

Wird jede Klasse in einer eigenen Datei definiert (=Konvention), so müssen Klassen vor der Verwendung in einer anderen Datei geladen werden.

Der in "index.php" definierte Autoload-Mechanismus automatisiert dies und erspart so viel Schreibarbeit. Voraussetzung ist die Befolgung der Konvention.

```php
function __autoload($class_name) {
    require_once $class_name . '.php';
}
```

Beispiel zum Autoloading

Gegeben Sei die Anweisung (siehe "index.php")

```
$controller = new Controller();
```

Ist die Klasse noch nicht bekannt, so lädt die Autoload-Funktion automatisch die Datei "Controller.php".

Änderung ab PHP 8

Die Mechaniken in PHP zum Laden von Klassen wurden modifiziert. Die Funktion `__autoload` gibt es so nicht mehr. Für PHP 8 lässt sich mit dem folgenden Beispiel jedoch das selbe erreichen:

```php
spl_autoload_register(function ($class_name) {
    require_once $class_name . '.php';
});
```

Zu beachten ist, dass die Implementierung der Funktion annahmen trifft, die ggf. nicht mehr zutreffen, wenn mit Unterordnern gearbeitet wird. Folgende Funktion ist eine weiterentwicklung, welche Namespaces berücksichtigt und den Speicherort von `start.php` als Bezug verwendet:

```php
spl_autoload_register(function($class) {
    $file = __DIR__ . '/' . str_replace('\\', '/', $class) . '.php';
    if (file_exists($file)) {
        require_once $file;
    }
});
```

ÜBUNG: MVC2-CONTROLLER-AUSWAHL

Wie kann in einer MVC2-Architektur der richtige Controller für eine HTTP-Anfrage ausgewählt werden?

- [ ] Das Model bestimmt den zuständigen Controller
- [ ] Über URL-Pfade mit URL-Rewriting (z.B. /products/list)
- [ ] Die View wählt den Controller direkt aus
- [ ] Über URL-Parameter (z.B. index.php?controller=products)

ÜBUNG: MVC2-KOMPONENTEN

Welche der folgenden Aufgaben gehört zu welcher MVC2-Komponente?

- [ ] View: SQL-Abfragen ausführen
- [ ] View: HTML-Templates mit Platzhaltern für dynamische Daten erstellen
- [ ] Controller: Formulardaten validieren und verarbeiten
- [ ] Model: Geschäftslogik und Datenbankzugriffe implementieren
- [ ] Controller: Ändert den View mittels DOM-API
- [ ] Model: HTTP-Parameter auswerten

ÜBUNG: MVC2-VIEW

Was sind typische Eigenschaften einer View in MVC2?

- [ ] Sie führt direkte Datenbankabfragen aus
- [ ] Sie verarbeitet POST-Daten selbständig
- [ ] Sie erhält ihre Daten vom Controller
- [ ] Sie enthält hauptsächlich HTML/CSS mit eingebetteten PHP-Ausgaben

ÜBUNG: VERWENDUNG DES MVC2

Erklären Sie, warum in einer MVC2-Webanwendung die Trennung von Model und View besonders wichtig ist. Geben Sie ein kurzes konkretes Beispiel.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt



6.6 ÜBUNGEN

ÜBUNG: PROTOKOLL

Erstelle eine Protokollierungsanwendung. Ein Formular mit einem Eingabefeld und einen Taster zu Absenden soll das Erfassen einer neuen Protokollnachricht ermöglichen, jede erfasste Nachricht soll in der aktuellen Sitzung mit einen Zeitvermerk gespeichert werden.

Nutzen Sie das bereitgestellte Projekt-Archiv php-protocol.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



Formular

- Erstelle ein einfaches Formular in HTML bestehend aus:
  - einem Label
  - einem Eingabefeld mit dem Namen `message`
  - einem Taster zum Absenden mit dem Namen `action-name` und dem Wert `save`



Verarbeitung

- Prüfe, ob in der Sitzung ein Index für die Protokollnachrichten existiert, andernfalls erstelle ein leeres Array (z.B. `messages`)
- *Hinweis: Mit der Funktion `isset` kann geprüft werden, ob eine Variable (oder ein Element in einem Array) gesetzt ist*
- Falls das Formular abgesendet wurde:
  - Beziehe die Nachricht
  - Prüfe ob die Nachricht nicht leer ist
  - Füge die Nachricht dem Array der Sitzung hinzu, und erweitere die Nachricht um eine Zeit und Datumsinformation
  - *Hinweis: Mit dem Funktionsaufruf `date("H:i:s d.m.Y")` kann das aktuelle Datum und die Uhrzeit als String erhalten werden*
- Gib alle bisher gesammelten Protokollinformationen in einer HTML-Liste aus

Erweiterung

Wie können die Informationen der Sitzung ausführlicher gespeichert werden? Aktuell wird ein einzelner String je Eintrag gespeichert. Wie lassen sich später einzelne Elemente wieder löschen?



Umbau der Anwendung zu einem MVC2

Bauen Sie die Anwendung nach dem MVC2 um. Ergänzen Sie einen Controller, identifizieren Sie das Modell und erstellen Sie den View.



Im Folgenden eine Musterlösung für das umgebaute Beispiel.

Nutzen Sie das bereitgestellte Projekt-Archiv php-mvc-protocol.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

ÜBUNG: POSITIONSSUCHE

Realisieren Sie eine Formulardatenverarbeitung, bei der Straße und Stadt eingegeben wird. Suchen Sie mittels OpenStreetMap nach den Geo-Coordinaten (Latitude / Longitude). Hierfür gibt es die Nominatim-API von OpenStreetMap, welche mittels Query-Parameter an einer vorgegebenen URL die Suche erlaubt.

Nutzen Sie das bereitgestellte Projekt-Archiv php-pos-search.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Eine Beispiel URL sieht wie folgt aus:

`https://nominatim.openstreetmap.org/search?street=esplanade&city=ingolstadt&`

Das Ergebnis ist ein JSON-Array, welches Objekte enthält, die mögliche Kartenpositionen beschreiben:

```
[
    {
        "lat": "48.7663189",
        "lon": "11.4325513",
        "display_name": "Reimanns, 10, Esplanade, Altstadt Südost, ..."
    },
    // ...
]
```



Zur Verfügung steht eine Funktion in PHP die einen HTTP-GET-Request absendet, die JSON-Antwort verarbeitet und bereits ein PHP-Array zurück liefert, andernfalls eine Exception:

```php
function simple_http_get($url) { /* ... */ }
try {
    $result = simple_http_get("https://....");
} catch(\Exception $e) {
    // ...
}
```

Aufgabe

- Implementieren Sie eine Formulardatenverarbeitung basierend auf der Vorlage. Ergänzen Sie notwendige Angaben im HTML-Teil, damit Sie Formulardaten verarbeiten können. Erzeugen Sie aus den Eingaben eine URL, bei der die Straße (street) und Stadt (city) in der URL korrekt eingesetzt wird. Speichern Sie das Ergebnis in einer Variable (z.B. $result).
- Füllen Sie das Formular mit den Eingaben erneut aus, falls diese bereits abgesendet wurden. Andernfalls können die Eingaben leer bleiben.
- Verarbeiten Sie das PHP-Array, welches Sie als Ergebnis aus dem Aufruf von simple_http_get erhalten. Iterieren Sie über dieses und geben Sie lat, lon und display_name in einer Liste aus.

Weitere Übung am Beispiel

- Erzeugen Sie mit der Ausgabe der Ergebnisse einen Link, der auf folgende URI verweist:

```
http://www.openstreetmap.org/?mlat=1234&mlon=1234&zoom=18
```

- Erweitern Sie Ihre Formulardatenverarbeitung so, dass immer Straße und Stadt angegeben werden muss. Marken Sie die Input-Felder, falls diese nicht korrekt ausgefüllt wurden mit der CSS-Klasse "is-invalid". Falls die Abfrage eine Exception erzeugt, geben Sie eine Information aus, dass es nicht geklappt hat.

- Verwenden Sie PHP-Session um vorherige Suchen geeignet zu speichern, so dass, wenn der Nutzer die Seite später wieder besucht die Eingaben von der letzten Suche noch ausgefüllt sind.
- Duplizieren Sie die Lösung, entfernen Sie PHP und versuchen Sie die Aufgabe mittels XmlHttpRequest und der DOM-API zu lösen. Achten Sie darauf, dass als HTTP-Request-Header der User-Agent auf einen Wert gesetzt wird.

```
var req = new XmlHttpRequest();
// ...
req.setRequestHeader('User-Agent', 'something');
// ...
```

